

CampusEats — Services, Contracts & Schema

CS543 Web Services · Assignment 2 · Design Document (Questions 1, 3, 4, 6)

Note: the submitted repository's `brief.md/README.md` were empty, so this design assumes the standard CampusEats scope — a campus food-ordering system with user accounts, restaurant/menu browsing, cart, order placement, payment, and delivery tracking. Adjust capabilities and owners below if your Assignment 1 brief specified something different.

Question 1 — Capabilities

Every distinct job the CampusEats system performs, expressed as verbs:

- 1 Register and authenticate a user
- 2 Manage user addresses (hostel/room, campus zone)
- 3 Browse restaurants and their menus
- 4 Manage a shopping cart (add / update / remove items)
- 5 Place an order
- 6 Process payment for an order
- 7 Assign and track a delivery
- 8 Notify a user of order, payment, and delivery status changes

Eight capabilities are listed (minimum required: six). Each capability is grouped by the data it reads or writes, which is what produces the six service boundaries used throughout this document.

Service ownership (capabilities grouped by data owned)

Service	Data owned	Team member	Capabilities covered
User Service	users, addresses	Ankur Gupta	Register/authenticate a user; manage addresses
Catalogue Service	restaurants, menu_items	Manish Saxena	Browse restaurants and menus
Order Service	carts, cart_items, orders, order_items	Curtains	Manage cart; place order (central operation)
Payment Service	payments	Sanu Swarnakar	Process payment for an order
Delivery Service	delivery_agents, deliveries	Shubhag Baluni	Assign and track a delivery
Notification Service	notifications	Ujjwal Sharma	Notify a user of status changes

Ownership above is a placeholder mapping (one service per teammate) — swap names to match your team's actual assignment before submitting.

Question 3 — Service Contracts

Each table lists the operations other services may call on that service. Only the operation, its inputs, outputs and error cases are exposed — no tables, no ID formats, no SQL.

Catalogue Service (main service — 4 operations)

Operation	Input	Output	Errors
listRestaurants	campusZone	list of {restaurantId, name, isOpen}	—
getMenu	restaurantId	list of {itemId, name, price, isAvailable}	RESTAURANT_NOT_FOUND
checkItemAvailability	restaurantId, items[{itemId, qty}]	{allAvailable, unitPrices, unavailableItems}	RESTAURANT_NOT_FOUND, RESTAURANT_CLOSED
decrementStock	restaurantId, items[{itemId, qty}]	{confirmed}	OUT_OF_STOCK, RESTAURANT_NOT_FOUND

Order Service (main service — 4 operations)

Operation	Input	Output	Errors
addToCart	userId, restaurantId, itemId, quantity	{cartId, itemCount}	ITEM_UNAVAILABLE, DIFFERENT_RESTAURANT_IN_CART
placeOrder	see full specification — Question 4	see full specification — Question 4	see full specification — Question 4
getOrderStatus	orderId, userId	{status, items, total, eta}	ORDER_NOT_FOUND, NOT_AUTHORIZED
cancelOrder	orderId, userId, reason	{refundInitiated}	ORDER_NOT_FOUND, ORDER_NOT_CANCELLABLE

User Service

Operation	Input	Output	Errors
getUserProfile	userId	{userId, name, phone}	USER_NOT_FOUND
validateAddress	addressId, userId	{valid, campusZone}	ADDRESS_NOT_FOUND, ADDRESS_NOT_OWNED

Payment Service

Operation	Input	Output	Errors
chargePayment	orderId, userId, amount, method	{paymentId, status}	PAYMENT_DECLINED, INVALID_METHOD, DUPLICATE_PAYMENT

Delivery Service

Operation	Input	Output	Errors
assignDelivery	orderId, restaurantZone, dropAddressId	{deliveryId, agentName, etaMinutes}	NO_AGENT_AVAILABLE, ZONE_NOT_SERVICED

Notification Service

Operation	Input	Output	Errors
sendNotification	userId, type, message	{notificationId, status}	USER_NOT_FOUND, CHANNEL_UNAVAILABLE

Question 4 — Full Specification: placeOrder

placeOrder is the central operation of CampusEats. It lives on the Order Service and orchestrates the Catalogue, User, Payment, Delivery, and Notification services to turn a cart into a confirmed order.

Inputs

- **userId** — the ordering user
- **cartId** — the user's active cart for one restaurant
- **deliveryAddressId** — which of the user's saved addresses to deliver to
- **paymentMethod** — UPI / CARD / WALLET
- **couponCode** (optional) — discount code

Outputs

- **orderId** — identifier of the newly created order
- **status** — PLACED
- **totalAmount** — item cost + delivery fee + tax – discount
- **estimatedDeliveryMinutes** — ETA returned by the Delivery Service

Error cases

Error	Meaning
USER_NOT_FOUND	userId does not resolve to a valid account
CART_EMPTY	The cart has no items at the time of the call
RESTAURANT_CLOSED	The restaurant is currently not accepting orders
ITEM_UNAVAILABLE	One or more cart items are no longer available or in stock
ADDRESS_INVALID	deliveryAddressId does not belong to this user
PAYMENT_FAILED	The Payment Service declined or could not process the charge
NO_AGENT_AVAILABLE	Order is placed but no delivery agent could be assigned immediately (queued for retry)
DUPLICATE_REQUEST	A retried call with the same idempotency key as an already-placed order

Internal details hidden from callers

Callers only ever see the inputs, outputs and error codes above. Everything below is internal to the Order Service's implementation of placeOrder and may change without affecting any other service:

- Re-fetching the cart's items and re-validating current price/availability against the Catalogue Service (checkItemAvailability, then decrementStock) — prices shown at add-to-cart time are not trusted blindly at order time
- Computing the final total (item prices + delivery fee + tax – coupon discount)
- Calling the User Service to confirm the delivery address belongs to this user (validateAddress)
- Calling the Payment Service to charge the order, and the compensating rollback (re-incrementing stock on the Catalogue Service) if the charge fails
- Calling the Delivery Service to assign an agent, and the fallback/retry queue used when no agent is immediately available

- Converting the cart into a persisted order: writing the orders and order_items rows, then clearing the cart
- Generating and checking an idempotency key so that a client's network retry does not create a second order
- Firing an asynchronous call to the Notification Service after the response is returned, so notification delivery never blocks or fails the order itself

Question 6 — Service Validation

Each service checked against the five properties of a service: reachable, self-contained, has a contract, independent, loosely coupled.

Service	Reachable	Self-contained	Has contract	Independent	Loosely coupled
User Service	Pass	Pass	Pass	Pass	Pass
Catalogue Service	Pass	Pass	Pass	Pass	Pass
Order Service	Pass	Pass	Pass	Pass	Pass
Payment Service	Pass	Pass	Pass	Pass	Pass
Delivery Service	Pass	Pass	Pass	Pass	Pass
Notification Service	Pass	FAIL (see note)	Pass	Pass	Pass

Failure noted: Notification Service — self-contained

The notifications table stores only the message, channel, type and status — it does not store a user's e-mail address or phone number (that data is owned by the User Service). To actually deliver a notification, Notification Service must synchronously call User Service's `getUserProfile` on every `sendNotification`. This means Notification Service cannot function if User Service is unreachable, even though notifications are otherwise an independent concern.

Fix

Have the User Service publish a lightweight event (or webhook) whenever a user's contact details are created or changed, and let Notification Service cache the current e-mail/phone/channel preference in its own table. `sendNotification` then reads its local cache instead of calling User Service on every request, restoring self-containment while keeping the two services loosely coupled through an event-based contract rather than a shared database.